

```

import pandas as pd

import matplotlib.pyplot as plt

# 1. Read the header separately to get metadata

# We read only the first 25 rows to get the settings

header_df = pd.read_csv("DS0001.CSV", nrows=25, header=None, usecols=[0, 1])

metadata = dict(zip(header_df[0], header_df[1]))

# Extract specific details for the plot

source = metadata.get('Source', 'Unknown CH')

v_scale = metadata.get('Vertical Scale', 'N/A')

h_scale = metadata.get('Horizontal Scale', 'N/A')

sample_rate = metadata.get('Sampling Period', 'N/A')

# 2. Read the actual waveform data

df = pd.read_csv("DS0001.CSV", skiprows=26, header=None, usecols=[0, 1])

df.columns = ['Time', 'Voltage']

# 3. Create the Plot

plt.figure(figsize=(12, 6))

plt.plot(df['Time'], df['Voltage'], color='tab:blue', linewidth=1, label=f'Signal ({source})')

# Add the details to the Plot

plt.title(f'Oscilloscope Trace: {source}', fontsize=14, fontweight='bold')

plt.xlabel(f'Time (Seconds) | Period: {sample_rate}s', fontsize=10)

plt.ylabel(f'Voltage (V) | Scale: {v_scale}V/div', fontsize=10)

# Add a text box with extra details in the corner

info_text = f"Mode: {metadata.get('Mode', 'N/A')}\nFirmware: {metadata.get('Firmware', 'N/A')}\nTrigger: {metadata.get('Trigger Level', 'N/A')}"

plt.annotate(info_text, xy=(0.02, 0.95), xycoords='axes fraction',

            bbox=dict(boxstyle="round", fc="white", ec="gray", alpha=0.8),

            verticalalignment='top')

plt.grid(True, which='both', linestyle='--', alpha=0.5)

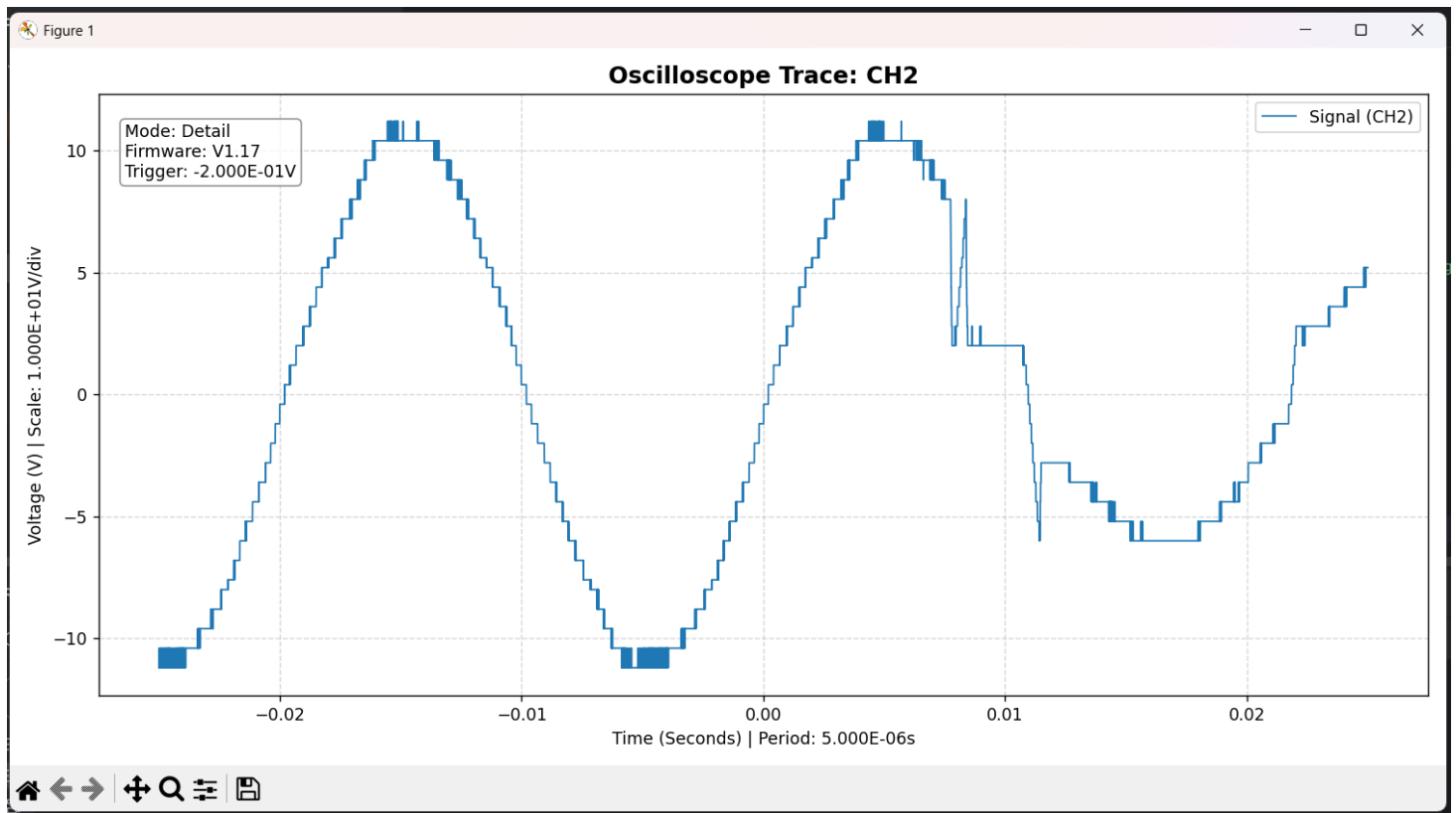
plt.legend(loc='upper right')

plt.tight_layout()

# 4. Show it!

plt.show()

```



Time Voltage

6599	0.008000	2.8
6600	0.008005	2.8
6601	0.008010	2.8
6602	0.008015	2.8
6603	0.008020	2.8
6604	0.008025	2.8
6606	0.008035	3.6
6607	0.008040	3.6
6608	0.008045	3.6
6609	0.008050	3.6

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```
# 1. Load Data
df = pd.read_csv("DS0001.CSV", skiprows=26, header=None, usecols=[0, 1])
df.columns = ['Time', 'Voltage']
sampling_period = 5e-06
```

```
def get_exact_fft(dataframe):
    # Removing DC offset to ensure the 50Hz peak is the primary focus
    sig = dataframe['Voltage'].values - np.mean(dataframe['Voltage'].values)
    n = len(sig)
```

```
# Raw FFT without windowing or padding
fft_vals = np.fft.rfft(sig)
```

```

fft_mag = np.abs(fft_vals) * (2.0 / n)
freqs = np.fft.rfftfreq(n, d=sampling_period)
return freqs, fft_mag

def print_exact_coordinates(label, freqs, magnitudes):
    # Defining target frequencies for the table
    target_freqs = [50, 100, 150, 200, 250, 300, 350, 400, 450, 500]
    total_mag = np.sum(magnitudes)

    print(f"\n--- EXACT COORDINATES: {label} ---")
    print(f"{'Target (Hz)':<12} | {'Actual (Hz)':<12} | {'Mag (V)':<12} | {'Contrib (%)':<12}")
    print("-" * 60)

    for t in target_freqs:
        # Find the index of the closest frequency bin in the raw data
        idx = np.abs(freqs - t).argmin()
        actual_f = freqs[idx]
        mag = magnitudes[idx]
        contribution = (mag / total_mag) * 100
        print(f"{'t':<12} | {'actual_f:<12.1f'} | {'mag:<12.4f'} | {'contribution:<12.2f}%")

```

```

# 2. Process Data
f_all, m_all = get_exact_fft(df)
print_exact_coordinates("FULL ORIGINAL SIGNAL", f_all, m_all)

# 3. Plotting (2 Rows, 1 Column)
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 12), layout="constrained")

```

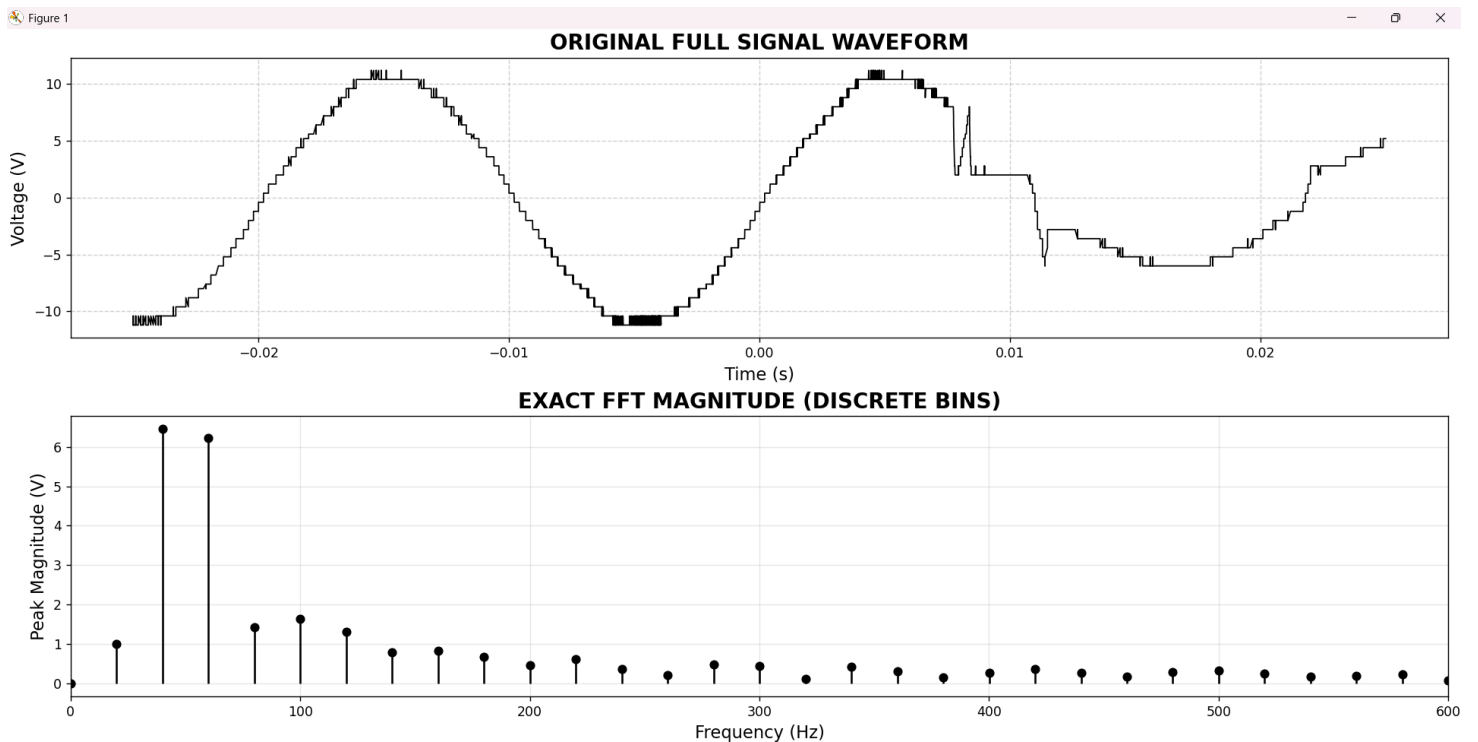
```

# Plot 1: Original Waveform
ax1.plot(df['Time'], df['Voltage'], color='black', linewidth=1)
ax1.set_title("ORIGINAL FULL SIGNAL WAVEFORM", fontsize=16, fontweight='bold')
ax1.set_ylabel("Voltage (V)", fontsize=13)
ax1.set_xlabel("Time (s)", fontsize=13)
ax1.grid(True, linestyle='--', alpha=0.6)

# Plot 2: Exact FFT Magnitude
# Using stem ensures you see the raw discrete frequency bins
ax2.stem(f_all, m_all, linefmt='black', markerfmt='ko', basefmt=' ')
ax2.set_xlim(0, 600) # Focus on the 0-600Hz range
ax2.set_title("EXACT FFT MAGNITUDE (DISCRETE BINS)", fontsize=16, fontweight='bold')
ax2.set_ylabel("Peak Magnitude (V)", fontsize=13)
ax2.set_xlabel("Frequency (Hz)", fontsize=13)
ax2.grid(True, alpha=0.3)

plt.show()

```



--- EXACT COORDINATES: FULL ORIGINAL SIGNAL ---

Target (Hz) | Actual (Hz) | Mag (V) | Contrib (%)

50	40.0	6.4603	10.53	%
100	100.0	1.6326	2.66	%
150	140.0	0.7886	1.29	%
200	200.0	0.4656	0.76	%
250	240.0	0.3623	0.59	%
300	300.0	0.4453	0.73	%
350	340.0	0.4308	0.70	%
400	400.0	0.2722	0.44	%
450	440.0	0.2634	0.43	%
500	500.1	0.3291	0.54	%

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy import signal

# 1. Load Data
df = pd.read_csv("DS0001.CSV", skiprows=26, header=None, usecols=[0, 1])
df.columns = ['Time', 'Voltage']
sampling_period = 5e-06
fs = 1 / sampling_period

# 2. Extract Full Dataset PSD
def get_psd_data(volt_data):
    # Removing DC component to prevent the 0Hz bin from squashing the scale
    sig = volt_data - np.mean(volt_data)
    # Using window=None and no padding ensures the raw, exact periodogram
    f, p = signal.periodogram(sig, fs, window=None, detrend=False)
    mask = f <= 600
    return f[mask], p[mask]

def print_contributions(label, freqs, psd_vals):
    target_freqs = [50, 100, 150, 200, 250, 300]
    total_power = np.sum(psd_vals)

    print(f"\n--- EXACT POWER CONTRIBUTION: {label} ---")
    print(f"{'Freq (Hz)':<10} | {'Power (V^2/Hz)':<15} | {'Contribution (%)':<15}")
    print("-" * 50)

    for target in target_freqs:
        idx = np.abs(freqs - target).argmin()
        actual_f = freqs[idx]
        power = psd_vals[idx]
        percentage = (power / total_power) * 100
        print(f"{'actual_f':<10.1f} | {'power':<15.2e} | {'percentage':<15.2f}%")

# 3. Process Data
f_all, p_all = get_psd_data(df['Voltage'])
print_contributions("FULL SIGNAL", f_all, p_all)

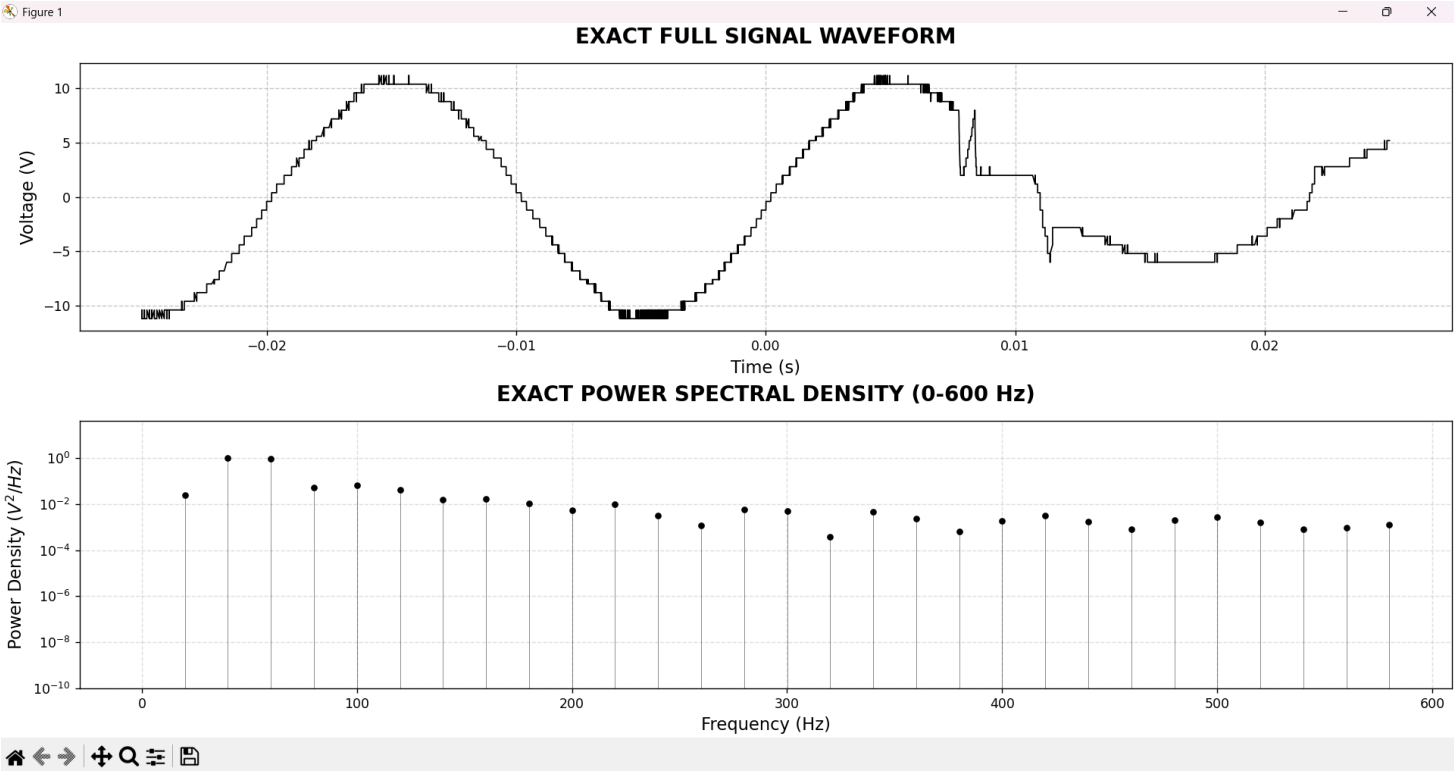
# 4. Plotting (Enhanced Clarity)
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 12), layout="constrained")

# --- Exact Waveform ---
ax1.plot(df['Time'], df['Voltage'], color='black', linewidth=1)
ax1.set_title("EXACT FULL SIGNAL WAVEFORM", fontsize=16, fontweight='bold', pad=15)
ax1.set_ylabel("Voltage (V)", fontsize=13)
ax1.set_xlabel("Time (s)", fontsize=13)
ax1.grid(True, linestyle='--', alpha=0.7)
```

```
# --- Exact PSD Stem Plot ---
# Using stem instead of plot shows the exact data point at each frequency bin
markerline, stemlines, baseline = ax2.stem(f_all, p_all, linefmt='black', basefmt=" ", markerfmt='ko')
plt.setp(markerline, markersize=4)
plt.setp(stemlines, linewidth=0.5, alpha=0.5)

ax2.set_yscale('log') # Log scale is necessary to see harmonics and fundamental together
ax2.set_ylim(bottom=1e-10) # Cut off noise floor for visibility
ax2.set_title("EXACT POWER SPECTRAL DENSITY (0-600 Hz)", fontsize=16, fontweight='bold', pad=15)
ax2.set_ylabel("Power Density (V^2/Hz)", fontsize=13)
ax2.set_xlabel("Frequency (Hz)", fontsize=13)
ax2.grid(True, which="both", linestyle='--', alpha=0.4)

plt.show()
```



--- EXACT POWER CONTRIBUTION: FULL SIGNAL ---

Freq (Hz) | Power (V^2/Hz) | Contribution (%)

40.0	1.04e+00	45.37	%
100.0	6.66e-02	2.90	%
140.0	1.55e-02	0.68	%
200.0	5.42e-03	0.24	%
240.0	3.28e-03	0.14	%
300.0	4.96e-03	0.22	%

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# 1. Load Data
df = pd.read_csv("DS0001.CSV", skiprows=26, header=None, usecols=[0, 1])
df.columns = ['Time', 'Voltage']
sampling_period = 5e-06
fs = 1 / sampling_period

# 2. Exact Trimming (1 cycle = 0.02s)
part1 = df[(df['Time'] >= -0.02) & (df['Time'] < 0)].copy()
part2 = df[(df['Time'] >= 0.005) & (df['Time'] < 0.025)].copy()

def get_exact_fft(df_segment):
    sig = df_segment['Voltage'].values
    sig = sig - np.mean(sig) # Remove DC
    n = len(sig)

    # Standard FFT without padding
    fft_vals = np.fft.rfft(sig)
    fft_mag = np.abs(fft_vals) * (2.0 / n)
    freqs = np.fft.rfftfreq(n, d=sampling_period)
    return freqs, fft_mag

def print_fft_coordinates(label, freqs, magnitudes):
    target_freqs = [50, 100, 150, 200, 250, 300, 350, 400, 450, 500]
    total_mag = np.sum(magnitudes)

    print(f"\n--- FFT COORDINATES & CONTRIBUTION: {label} ---")
    print(f"{'Freq (Hz)':<10} | {'Magnitude (V)':<15} | {'Contribution (%)':<15}")
    print(" " * 50)

    for target in target_freqs:
        # Find the index of the closest frequency bin
        idx = np.abs(freqs - target).argmin()
        actual_f = freqs[idx]
        mag = magnitudes[idx]
        contribution = (mag / total_mag) * 100
        print(f"{'actual_f:<10.1f} | {mag:<15.4f} | {contribution:<15.2f}%"")

# 3. Process and Print Table Data
f1, m1 = get_exact_fft(part1)
f2, m2 = get_exact_fft(part2)

print_fft_coordinates("HEALTHY SEGMENT", f1, m1)
print_fft_coordinates("FAULTY SEGMENT", f2, m2)

# 4. Plotting
fig, axes = plt.subplots(2, 2, figsize=(15, 12), layout="constrained")

# --- Healthy ---
axes[0, 0].plot(part1['Time'], part1['Voltage'], color='green')
axes[0, 0].set_title("WAVEFORM: HEALTHY (1 CYCLE)", fontweight='bold')
axes[0, 0].grid(True, alpha=0.3)

axes[1, 0].stem(f1, m1, linefmt='green', markerfmt='go', basefmt=' ')
axes[1, 0].set_xlim(0, 550)
axes[1, 0].set_title("FFT: HEALTHY MAGNITUDE", fontweight='bold')
axes[1, 0].set_ylabel("Peak Voltage (V)")
axes[1, 0].set_xlabel("Frequency (Hz)")

# --- Faulty ---
axes[0, 1].plot(part2['Time'], part2['Voltage'], color='red')
axes[0, 1].set_title("WAVEFORM: FAULTY (1 CYCLE)", fontweight='bold')
axes[0, 1].grid(True, alpha=0.3)

axes[1, 1].stem(f2, m2, linefmt='red', markerfmt='ro', basefmt=' ')
axes[1, 1].set_xlim(0, 550)
axes[1, 1].set_title("FFT: FAULTY MAGNITUDE", fontweight='bold')
axes[1, 1].set_ylabel("Peak Voltage (V)")
axes[1, 1].set_xlabel("Frequency (Hz)")

plt.show()
```



--- FFT COORDINATES & CONTRIBUTION: HEALTHY SEGMENT ---

Freq (Hz) | Magnitude (V) | Contribution (%)

49.9	10.9006	46.75	%
99.8	0.0826	0.35	%
149.7	0.0362	0.16	%
199.6	0.0206	0.09	%
249.4	0.1335	0.57	%
299.3	0.0640	0.27	%
349.2	0.1260	0.54	%
399.1	0.0393	0.17	%
449.0	0.0699	0.30	%
498.9	0.0162	0.07	%

--- FFT COORDINATES & CONTRIBUTION: FAULTY SEGMENT ---

Freq (Hz) | Magnitude (V) | Contribution (%)

50.1	6.9520	23.16	%
100.2	1.4846	4.95	%
150.3	0.7357	2.45	%
200.5	0.6428	2.14	%
250.6	0.9048	3.01	%
300.7	0.5456	1.82	%
350.8	0.1803	0.60	%
400.9	0.4454	1.48	%
451.0	0.2246	0.75	%
501.1	0.1295	0.43	%

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy import signal

# 1. Load Data
df = pd.read_csv("DS0001.CSV", skiprows=26, header=None, usecols=[0, 1])
df.columns = ['Time', 'Voltage']
sampling_period = 5e-06
fs = 1 / sampling_period

# 2. Extract Segments
healthy_seg = df[(df['Time'] >= -0.02) & (df['Time'] < 0)].copy()
faulty_seg = df[(df['Time'] >= 0.005) & (df['Time'] < 0.025)].copy()

def get_psd_data(volt_data):
    sig = volt_data - np.mean(volt_data)
    f, p = signal.periodogram(sig, fs, window=None, detrend=False)
    mask = f <= 600
    return f[mask], p[mask]

def print_contributions(label, freqs, psd_vals):
    target_freqs = [50, 100, 150, 200, 250, 300]
    total_power = np.sum(psd_vals)

    print(f"\n--- POWER CONTRIBUTION: {label} ---")
    print(f"{'Freq (Hz)':<10} | {'Power (V^2/Hz)':<15} | {'Contribution (%)':<15}")
    print("-" * 45)

    for target in target_freqs:
        # Find the index of the closest frequency in the PSD array
        idx = np.abs(freqs - target).argmin()
        actual_f = freqs[idx]
        power = psd_vals[idx]
        percentage = (power / total_power) * 100
        print(f"{'actual_f:<10.1f} | {'power:<15.2e} | {'percentage:<15.2f}%")

# 3. Calculate and Print Stats
f_h, p_h = get_psd_data(healthy_seg['Voltage'])
f_f, p_f = get_psd_data(faulty_seg['Voltage'])

print_contributions("HEALTHY CYCLE", f_h, p_h)
print_contributions("FAULTY CYCLE", f_f, p_f)

# 4. Plotting (Kept standard for visual reference)
```

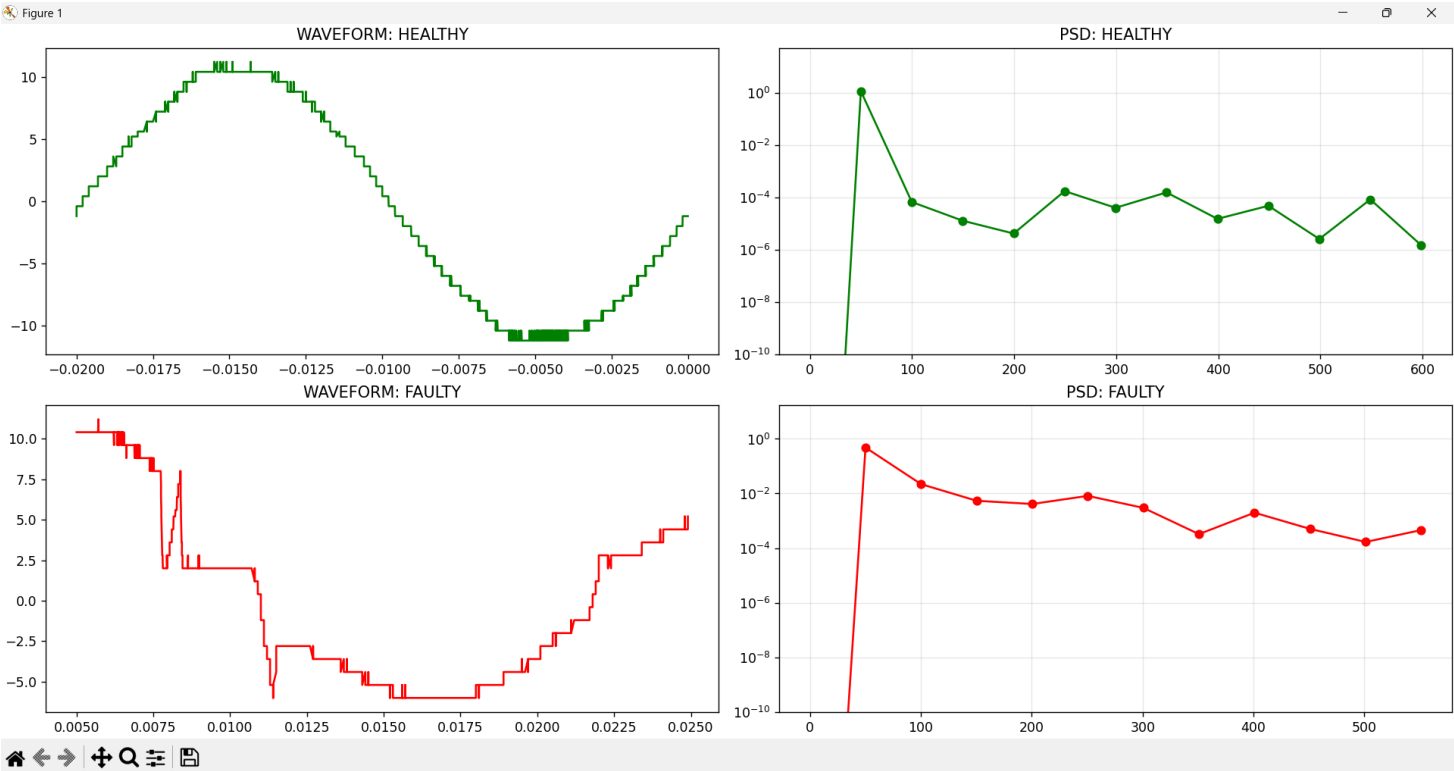
```
fig, axes = plt.subplots(2, 2, figsize=(15, 12), layout="constrained")

def plot_analysis(row, f, p, time_data, volt_data, title, color):
    axes[row, 0].plot(time_data, volt_data, color=color)
    axes[row, 0].set_title(f"WAVEFORM: {title}")

    axes[row, 1].semilogy(f, p, color=color, marker='o')
    axes[row, 1].set_ylim(bottom=1e-10)
    axes[row_idx := row, 1].set_title(f"PSD: {title}")
    axes[row, 1].grid(True, which="both", alpha=0.3)

plot_analysis(0, f_h, p_h, healthy_seg['Time'], healthy_seg['Voltage'], "HEALTHY", 'green')
plot_analysis(1, f_f, p_f, faulty_seg['Time'], faulty_seg['Voltage'], "FAULTY", 'red')

plt.show()
```



--- POWER CONTRIBUTION: HEALTHY CYCLE --- (particular freq wrt others)

Freq (Hz) | Power (V^2/Hz) | Contribution (%)

49.9	1.19e+00	99.95	%
99.8	6.84e-05	0.01	%
149.7	1.31e-05	0.00	%
199.6	4.26e-06	0.00	%
249.4	1.79e-04	0.01	%
299.3	4.10e-05	0.00	%

--- POWER CONTRIBUTION: FAULTY CYCLE ---

Freq (Hz) | Power (V^2/Hz) | Contribution (%)

50.1	4.82e-01	91.28	%
100.2	2.20e-02	4.16	%
150.3	5.40e-03	1.02	%
200.5	4.12e-03	0.78	%
250.6	8.17e-03	1.55	%
300.7	2.97e-03	0.56	%

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# 1. Load Data
# (Assuming CSV is in the same directory)
df = pd.read_csv("DS0001.CSV", skiprows=26, header=None, usecols=[0, 1])
df.columns = ['Time', 'Voltage']
sampling_period = 5e-06

# 2. Extract Exact Segments
healthy_seg = df[(df['Time'] >= -0.02) & (df['Time'] < 0)].copy()
faulty_seg = df[(df['Time'] >= 0.005) & (df['Time'] < 0.025)].copy()

def calculate_exact_fft(segment, period):
    sig = segment['Voltage'].values - np.mean(segment['Voltage'].values)
    n = len(sig)
    fft_vals = np.fft.rfft(sig)
    fft_mag = np.abs(fft_vals) * (2.0 / n)
    freqs = np.fft.rfftfreq(n, d=period)
    return freqs, fft_mag

# 3. Create the Comparison Plot
# Using layout="constrained" automatically manages spacing better than tight_layout
fig, axes = plt.subplots(3, 2, figsize=(14, 12), layout="constrained")

# --- ROW 1: COMPLETE DATA ---
axes[0, 0].plot(df['Time'], df['Voltage'], color='black', linewidth=1)
axes[0, 0].set_title("Original Complete CSV Waveform", fontweight='bold')
f_all, m_all = calculate_exact_fft(df, sampling_period)
axes[0, 1].stem(f_all[(f_all <= 600)], m_all[(f_all <= 600)], linefmt='black', basefmt=' ')
axes[0, 1].set_title("FFT of Full Data (Shows Leakage)", fontweight='bold')

# --- ROW 2: HEALTHY CYCLE ---
axes[1, 0].plot(healthy_seg['Time'], healthy_seg['Voltage'], color='green')
axes[1, 0].set_title("Healthy Segment (Exactly 1 Cycle)", fontweight='bold')
f_h, m_h = calculate_exact_fft(healthy_seg, sampling_period)
axes[1, 1].stem(f_h[(f_h <= 600)], m_h[(f_h <= 600)], linefmt='green', basefmt=' ')
axes[1, 1].set_title("Exact FFT: Healthy (50Hz Peak)", fontweight='bold')

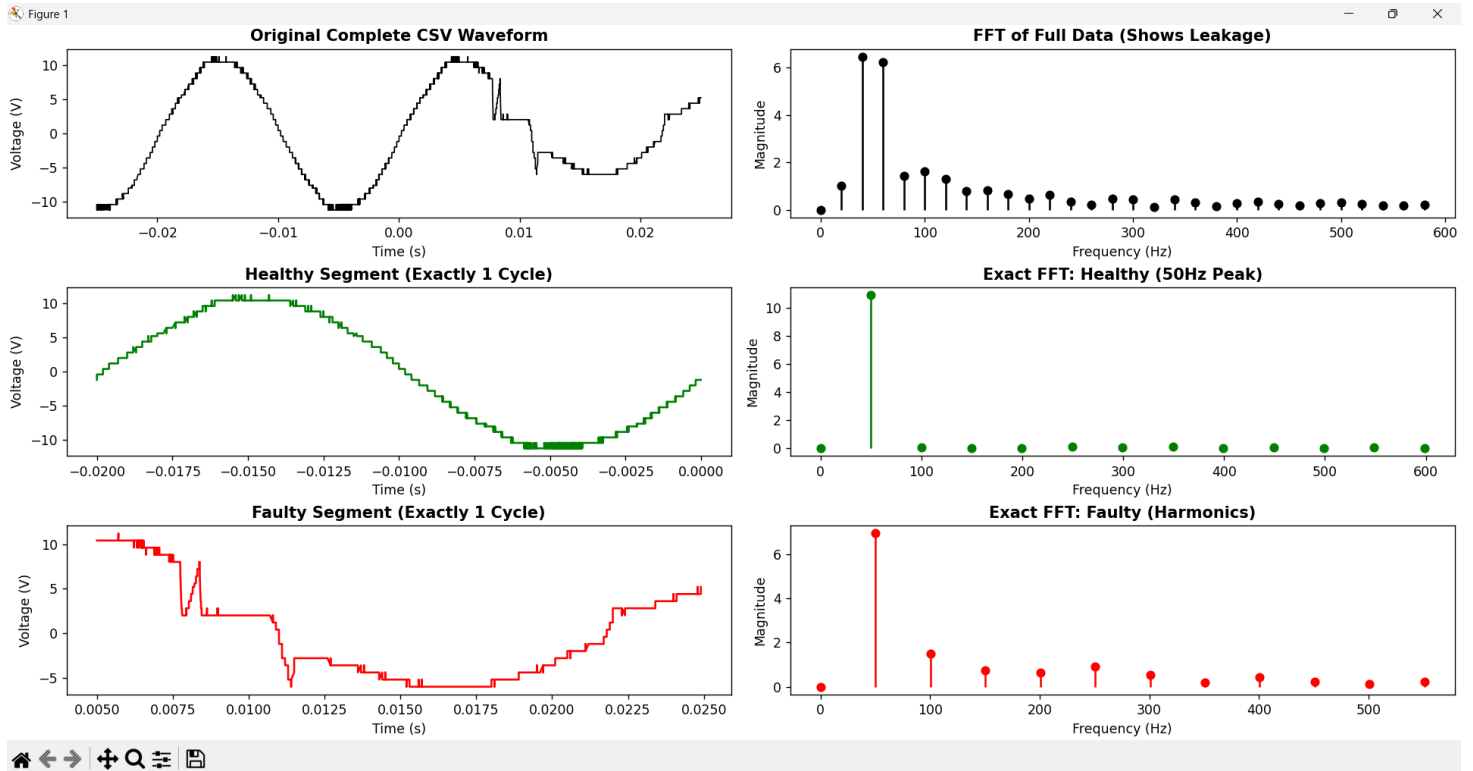
# --- ROW 3: FAULTY CYCLE ---
axes[2, 0].plot(faulty_seg['Time'], faulty_seg['Voltage'], color='red')
axes[2, 0].set_title("Faulty Segment (Exactly 1 Cycle)", fontweight='bold')
f_f, m_f = calculate_exact_fft(faulty_seg, sampling_period)
axes[2, 1].stem(f_f[(f_f <= 600)], m_f[(f_f <= 600)], linefmt='red', basefmt=' ')
axes[2, 1].set_title("Exact FFT: Faulty (Harmonics)", fontweight='bold')

# --- Formatting Labels ---
for ax in axes[:, 0]:
    ax.set_ylabel("Voltage (V)")
    ax.set_xlabel("Time (s)")

for ax in axes[:, 1]:
    ax.set_xlabel("Frequency (Hz)")
    ax.set_ylabel("Magnitude")

# If labels still feel tight, you can manually nudge the vertical spacing:
# plt.subplots_adjust(hspace=0.4)

plt.show()
```



```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy import signal
```

```
# 1. Load Data
df = pd.read_csv("DS0001.CSV", skiprows=26, header=None, usecols=[0, 1])
df.columns = ['Time', 'Voltage']
sampling_period = 5e-06
fs = 1 / sampling_period
```

```
# 2. Extract Segments
healthy_seg = df[(df['Time'] >= -0.02) & (df['Time'] < 0)].copy()
faulty_seg = df[(df['Time'] >= 0.005) & (df['Time'] < 0.025)].copy()
```

```
# 3. Create the Comparison Plot
# Increased height to 18 to ensure no overlapping
fig, axes = plt.subplots(3, 2, figsize=(15, 18), layout="constrained")
```

```
def plot_row(row_idx, time_data, volt_data, title_suffix, color):
    sig = volt_data - np.mean(volt_data)
```

```
# --- WAVEFORM ---
axes[row_idx, 0].plot(time_data, volt_data, color=color, linewidth=1.2)
axes[row_idx, 0].set_title(f"WAVEFORM: {title_suffix.upper()}", fontsize=16, pad=20, fontweight='bold')
axes[row_idx, 0].set_ylabel("Voltage (V)", fontsize=13)
axes[row_idx, 0].set_xlabel("Time (seconds)", fontsize=13)
axes[row_idx, 0].grid(True, linestyle='--', alpha=0.7)
```

```
# --- PSD ---
f_psd, p_den = signal.periodogram(sig, fs, window=None, detrend=False)
mask_psd = f_psd <= 600
```

```
axes[row_idx, 1].semilogy(f_psd[mask_psd], p_den[mask_psd], color=color, marker='o', markersize=4)
axes[row_idx, 1].set_ylim(bottom=1e-10)
```

```
axes[row_idx, 1].set_title(f"PSD: {title_suffix.upper()}", fontsize=16, pad=20, fontweight='bold')
axes[row_idx, 1].set_xlabel("Frequency (Hz)", fontsize=13)
axes[row_idx, 1].set_ylabel("Power Density ($V^2/Hz$)", fontsize=13)
axes[row_idx, 1].grid(True, which="both", linestyle='--', alpha=0.5)
```

```
# Execute Plotting
plot_row(0, df['Time'], df['Voltage'], "Full Dataset", 'black')
plot_row(1, healthy_seg['Time'], healthy_seg['Voltage'], "Healthy Cycle", 'green')
plot_row(2, faulty_seg['Time'], faulty_seg['Voltage'], "Faulty Cycle", 'red')
```

```
# Final spacing safety check
plt.show()
```

